

# Optimizacija več-agentnega iskanja poti z uvedbo lokalnih vodij

Andraž Šošterič  
andraz.sosteric@student.um.si  
Faculty of Electrical  
Engineering and Computer Science,  
University of Maribor  
Koroška cesta 46  
SI-2000 Maribor, Slovenia

Nejc Fras  
nejc.fras@student.um.si  
Faculty of Electrical  
Engineering and Computer Science,  
University of Maribor  
Koroška cesta 46  
SI-2000 Maribor, Slovenia

David Lipavec  
david.lipavec@student.um.si  
Faculty of Electrical  
Engineering and Computer Science,  
University of Maribor  
Koroška cesta 46  
SI-2000 Maribor, Slovenia

Tadej Teršek  
tadej.tersek@student.um.si  
Faculty of Electrical  
Engineering and Computer Science,  
University of Maribor  
Koroška cesta 46  
SI-2000 Maribor, Slovenia

## KLJUČNE BESEDE

centralizirano načrtovanje poti, distribuirano načrtovanje poti, usklajevanje agentov, izogibanje trkom, optimizacija poti

## 1 UVOD

V tem članku obravnavamo problem več-agentnega iskanja poti (Multi-Agent Path Finding / MAPF), ki se v praksi pojavlja v številnih domenah, kot so robotizirana skladišča, avtonomna vozila, inteligentni transportni sistemi in drugi logistični sistemi. Gre za temeljni problem koordinacije več avtonomnih enot v skupnem prostoru, kjer mora vsak agent doseči svoj cilj brez trkov z drugimi agenti ali ovirami.

Cilj problema je načrtovati poti za množico agentov v diskretnem prostoru tako, da se izognemo medsebojnim konfliktom in hkrati optimiziramo določene metrike, kot sta skupni čas izvajanja (ang. *makespan*) ali vsota dolžin poti (ang. *sum of costs*) [8].

Repozitorij z izvorno kodo projekta je na voljo na našem github repozitoriju<sup>1</sup>.

V nadaljevanju bomo primerjali centralizirane metode, katerih predstavnik sta LaCAM in PBS, ki načrtujejo poti za vse agente z uporabo globalnega pogleda na problem, ter decentralizirane pristope, kot sta Follower in MATS-LP, kjer posamezni agenti sprejemajo odločitve lokalno brez centralnega nadzora in imajo pregled le nad svojo okolico.

Na ta način bomo analizirali ključne razlike med obema paradigama z vidika učinkovitosti, skalabilnosti in kakovosti rešitev. Centralizirani pristopi praviloma zagotavljajo bolj optimalne rešitve, vendar slabše skalirajo z naraščajočim številom agentov, medtem ko decentralizirani pristopi omogočajo boljše razširljivost, vendar pogosto na račun kakovosti rešitev.

Kot glavni referenčni algoritem izberemo delo *LaCAM: Search-Based Algorithm for Quick Multi-Agent Pathfinding* [5], ki predstavlja sodoben in učinkovit centraliziran pristop k reševanju problema MAPF. Izbrani članek služi kot referenčna osnova za našo analizo,

saj ponuja dobro ravnovesje med učinkovitostjo in skalabilnostjo ter omogoča relativno enostavno reprodukcijo eksperimentov.

LaCAM bomo uporabili kot predstavnika centraliziranih metod, s katerim bomo primerjali decentralizirane pristope, kot sta Follower in MATS-LP, ter tako sistematično analizirali razlike med globalnim in lokalnim načrtovanjem poti.

### 1.1 Definicija problema

Stern et al. [8] definirajo problem več-agentnega iskanja poti (Multi-Agent Path Finding - MAPF) kot trojico

$$\langle G, s, t \rangle,$$

kjer je  $G = (V, E)$  neusmerjen graf, pri čemer  $V$  predstavlja množico vozlišč,  $E$  pa množico povezav.

Podana je množica agentov  $A = \{1, 2, \dots, k\}$ . Funkcija

$$s : \{1, \dots, k\} \rightarrow V$$

preslika vsakega agenta v njegovo začetno vozlišče, funkcija

$$t : \{1, \dots, k\} \rightarrow V$$

pa v njegovo ciljno vozlišče.

Naj bo  $\pi_i[t] \in V$  položaj agenta  $i$  v diskretnem času  $t \in \mathbb{N}_{\geq 0}$ . Pri vsakem koraku  $t$  se agent  $i$  lahko premakne v sosednje vozlišče ali ostane na trenutnem, tj.

$$\pi_i[t+1] \in \text{neigh}(\pi_i[t]) \cup \{\pi_i[t]\},$$

kjer je  $\text{neigh}(v)$  množica vozlišč, sosednjih vozlišču  $v \in V$  [5].

Po Okamuri [5] se mora agent izogibati naslednjim vrstam konfliktov:

- **Vozliščni konflikt:** Do vozliščnega konflikta (ang. vertex conflict) pride, če nameravata  $\pi_i$  in  $\pi_j$  zasedeti isto vozlišče v istem trenutku, torej obstaja časovni korak  $t$ , da velja  $\pi_i[t] = \pi_j[t]$ . [8]
- **Zamenjalni konflikt:** Do zamenjalnega konflikta (ang. swap conflict) pride, če nameravata  $\pi_i$  in  $\pi_j$  zamenjati svoji poziciji v istem trenutku, torej obstaja časovni korak  $t$ , da velja  $\pi_i[t] = \pi_j[t+1] \wedge \pi_i[t+1] = \pi_j[t]$  [8].

<sup>1</sup><https://github.com/Asos75/MultiAgent>

Rešitev MAPF za instance, omenjene zgoraj, je množica brezkonfliktnih poti  $\{\pi_1, \dots, \pi_k\}$  takih, da obstaja časovni korak  $T \in \mathbb{N}_{\geq 0}$ , kjer agent doseže cilj. [5]

## 2 SORODNA DELA

V tem razdelku izpostavimo ključna dela s področja več-agentnega iskanja poti, ki dopolnjujejo in razširjajo pogled iz uvoda. Pri vsakem delu na kratko povemo, kaj je njegova glavna ideja in kakšen doprinos ima k našemu eksperimentu.

### 2.1 Centralizirani pristopi

*EECBS* [4] je nadgradnja klasičnega CBS, ki ohrani dvonivojsko iskanje, hkrati pa z *Explicit Estimation Search* in uteževanjem omogoča nadzorovano suboptimalnost (uporabnik nastavi kompromis med kakovostjo in časom). Avtorji poročajo o bistveno boljši skalabilnosti na večjih instancah ob majhni, dokazljivo omejeni degradaciji kakovosti rešitve. *EECBS* uporabimo kot referenčno izhodišče za razumevanje kompromisa »hitrost vs. kvaliteta« pri centraliziranih metodah; njegove ideje utemeljijo, zakaj v evalvaciji poleg kakovosti (SoC/makespan) spremljamo tudi čas izvajanja in skalabilnost.

*CBSH2* [3] izboljša CBS z močnejšimi heuristikami in tehniko *disjoint splitting*, ki z uvedbo pozitivnih omejitev zmanjša podvajanje iskanja in s tem velikost iskalnega drevesa. Rezultat je hitrejšo reševanje in boljša skalabilnost ob ohranjeni rešitveni kakovosti tipične za CBS-družino. *CBSH2* obravnavamo kot dodatno referenco znotraj centraliziranih metod, ki pokaže, da so izboljšave pogosto posledica boljšega upravljanja konfliktov; to nam pomaga pri interpretaciji rezultatov *LaCAM*/*PBS* (kjer tudi ciljamo na učinkovitejšo obravnavo konfliktov), če opazimo razlike predvsem v času in ne v SoC.

### 2.2 Decentralizirani pristopi

*Follower* [1] je decentraliziran pristop za *lifelong* MAPF, ki kombinira (i) heuristični planer (npr.  $A^*$  nad statičnimi ovirami), ki generira »razpršene« poti z namenom zmanjšanja zasičenosti ozkih grl, in (ii) naučeno politiko sledenja, ki lokalno reagira na druge agente in se izogiba trkom. Izvajanje je povsem lokalno (brez centralnega koordinatorja), zato metoda dobro skalira, vendar je kakovost poti odvisna od lokalnih informacij in stabilnosti naučene politike. *Follower* je eden naših glavnih decentraliziranih primerjalnih algoritmov; z njim ovrednotimo, kako se učeni, lokalni pristopi primerjajo s centraliziranimi *LaCAM*/*PBS* in z našim hibridnim pristopom z lokalnimi vodji.

*Decentralized Monte Carlo Tree Search for Partially Observable Multi-agent Pathfinding* [7] predlaga decentralizirano reševanje *lifelong* MAPF pri delni opazljivosti: vsak agent iz lokalnih opazovanj zgradi intrinzični (lokalni) MDP in na njem izvaja MCTS, ki ga dodatno usmerja nevronska politika. S simulacijami v drevesu agenti ocenjujejo kratkoročne interakcije z bližnjimi agenti, koordinacija pa nastaja implicitno (brez eksplicitne komunikacije ali centralnega koordinatorja), kar vodi do dobre skalabilnosti. delo utemeljuje našo izbiro decentraliziranih primerjalnih metod na *POGEMA* (delna opazljivost) in je neposredno relevantno za komponento *MATS-LP* v naši primerjavi, kjer želimo videti, ali »lookahead« (*MCTS*) izboljša pretočnost in robustnost glede na bolj reaktivne politike.

*PRIMAL2* [2] je zgodnje, vplivno delo, ki pokaže, da je mogoče MAPF reševati z decentraliziranimi politikami na osnovi lokalnih opazovanj, naučenimi s kombinacijo *imitation learning* (iz centraliziranih planerjev) in *reinforcement learning*. Pristop zelo dobro skali, vendar pogosto žrtvuje optimalnost in lahko povzroča neusklajenosti v gostih scenarijih. *PRIMAL2* uporabljamo kot kontekst za interpretacijo rezultatov učenih decentraliziranih metod (*Follower*/*MATS-LP*/*SCRIMP*): če pri njih opazimo slabši SoC, je to pričakovani trade-off, ki ga literatura že izpostavlja.

*SCRIMP* [9] uvede eksplicitno komunikacijo med agenti z uporabo transformer arhitekture in pozornostnega mehanizma, kar omogoča učinkovitejšo koordinacijo tudi pri zelo omejenem vidnem polju. S tem pristop preseže omejitve povsem nekomunikacijskih lokalnih politik, hkrati pa ohrani decentralizirano izvajanje in dobro skalabilnost. *SCRIMP* služi kot motivacija za naš »hibridni« pogled (lokalni vodje): kaže, da lahko dodaten koordinacijski signal (komunikacija ali posredno usklajevanje) izboljša pretočnost; naš pristop to doseže z lažjim mehanizmom vodij, ki ga lahko primerjamo z relacijo med čisto lokalnimi in bolj koordiniranimi pristopi.

### 2.3 Primerjalne platforme

*POGEMA* [6] ponuja standardizirano platformo za evalvacijo MAPF algoritmov v (delno) opazljivih okoljih, z generatorjem instanc in enotnimi metrikami ter protokolom izvajanja. Omogoča pošteno in ponovljivo primerjavo različnih pristopov. *POGEMA* je naše glavno evalvacijsko ogrodje, zato se metodologija, metrike in primerjava rezultatov neposredno opirajo na to delo.

## 3 METODOLOGIJA

V tem poglavju bomo predstavili metodologijo vrednotenja in primerjanja izbranih algoritmov za problem večagentnega iskanja poti. Cilj eksperimentalnega dela je primerjati centralizirane pristope, kot sta *LaCAM* in *PBS*, z decentraliziranimi pristopi *Follower* in *MATS-LP* ter ovrednotiti predlagani hibridni pristop z lokalnimi vodji.

### 3.1 Experimentalno okolje

Za primerjalno okolje bomo uporabili knjižnico *POGEMA* [6], ki omogoča standardizirano evalvacijo MAPF algoritmov v delno opazljivih okoljih, ter to kombinirali z *MovingAI* [8] knjižnico zemljevidov za testiranje agentov.

*POGEMA* je dostopna kot standardna Python knjižnica, medtem ko so implementacije posameznih algoritmov pridobljene iz uradnih repozitorijev avtorjev izbranih člankov. Ker različni algoritmi uporabljajo različne vhodne formate in interne predstavitve okolja, je bilo potrebno za vsako implementacijo razviti poseben prilagoditveni sloj (*adapter*), ki omogoča vključitev algoritma v skupno evalvacijsko ogrodje.

### 3.2 Metrike

**Sum-of-Costs (SoC)** je vsota časovnih korakov, ki jih vsi agenti porabijo za doseg svojih ciljnih vozlišč [8]:

$$\text{SoC} = \sum_{i=1}^k |\pi_i|,$$

kjer je  $|\pi_i|$  dolžina poti agenta  $i$ . Nižja vrednost pomeni učinkovitejšo rešitev.

SoC je ena izmed najpogostejše uporabljenih metrik v MAPF literaturi [8], kar omogoča neposredno primerjavo z obstoječimi deli. Metrika meri skupni strošek kot vsoto dolžin poti vseh agentov in s tem neposredno zajame celostno učinkovitost sistema.

Metrika je še posebej primerna za one-shot scenarije, saj minimizacija SoC ustreza cilju minimizacije skupne porabe časa oziroma virov. Hkrati kaznuje neučinkovite poti posameznih agentov, kar spodbuja iskanje globalno učinkovitih rešitev. SoC bomo zato uporabili za vrednotenje algoritmov LaCAM in PBS na zbirki MovingAI.

**Makespan** je čas, ki ga porabi najdaljša pot med vsemi agenti, torej maksimum nad vsemi  $|\pi_i|$  [8]:

$$\text{makespan} = \max_{i \in \{1, \dots, k\}} |\pi_i|.$$

Metrika odraža skupno trajanje reševanja instance in je primerna za scenarije, kjer je ključen vzporedni zaključek vseh agentov. Makespan je uveljavljena metrika v MAPF literaturi [8] in je še posebej relevantna v praktičnih aplikacijah, kjer je ključen vzporedni zaključek vseh agentov.

Makespan je pri primerjavi centraliziranih algoritmov nujna dopolnitev k SoC, saj sama vsota stroškov namreč ne razkrije, ali posamezen agent morda bistveno zavlačuje celotno skupino. Z vključitvijo obeh metrik tako dobimo celovitejšo in bolj zanesljivo oceno učinkovitosti algoritma v realnih pogojih.

**Stopnja uspeha (Success Rate)** meri delež instanc, pri katerih vsi agenti uspešno dosežejo svoje cilje znotraj predvidenega časovnega horizonta [6].

Stopnja uspeha je primarna metrika platforme POGEMA in skupaj s pretokom (ang. throughput) tvori osnovo za vrednotenje na tej platformi [6]. Zasnovana je posebej za vrednotenje decentraliziranih metod v delno opazljivih okoljih. Metrika je za naš eksperiment ključna, saj za razliko od SoC in makespana meri tudi robustnost algoritma oziroma njegovo sposobnost, da instanco v celoti reši kljub omejenim lokalnim opazovanjem.

**Čas načrtovanja** merimo kot skupni čas izvajanja algoritma na posamezni instanci. Ta metrika je še posebej pomembna pri primerjavi med kategorijama, saj centralizirani načrtovalec tipično porabi več časa za globalno načrtovanje, kot decentralizirani agenti.

Čas načrtovanja je v praktičnih aplikacijah enako pomemben kot kakovost rešitve, saj algoritem z optimalno potjo, a dolgim časom načrtovanja, v realnih sistemih ni uporaben. Ta metrika nam tako omogoča primerjavo med pristopoma z vidika razširljivosti in praktične uporabnosti.

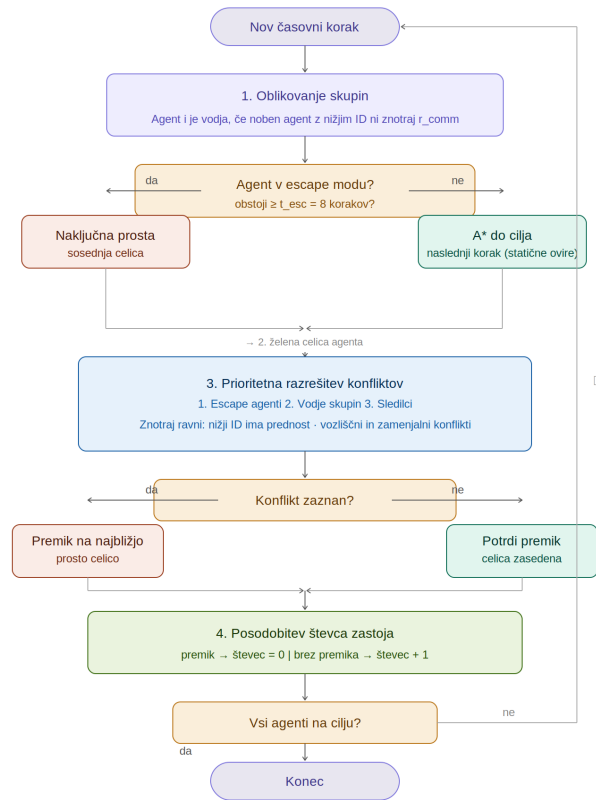
### 3.3 Protokol primerjave

Primerjavo bomo izvedli na dveh ravneh:

- (1) **Znotraj kategorije:** LaCAM in PBS primerjamo na MovingAI zbirki po metrikah SoC, makespan in čas načrtovanja. Follower in MATS-LP primerjamo na POGEMA zbirki po metrikah throughput in stopnja uspeha.
- (2) **Med kategorijama na skupni zbirki:** Obe kategoriji bomo primerjali med seboj na skupni zbirki, kjer bomo primerjali prej navedene metrike.

### 3.4 Uvedba lokalnih vodij

Glavni prispevek tega dela je predlagana nadgradnja obstoječih pristopov, ki uvaja koncept *lokalnih vodij*. Gre za hibridni algoritem *Local Leaders*, ki združuje prednosti centraliziranega in decentraliziranega pristopa. Algoritem je povsem *online* – brez vnapijšnjega načrtovanja – in deluje v petih korakih pri vsakem časovnem koraku (sl. 1).



**Slika 1: Diagram poteka algoritma Local Leaders: dinamično oblikovanje skupin, določitev vodje, A\* načrtovanje in prioriteta razrešitev konfliktov.**

**1. Oblikovanje skupin.** Na začetku vsakega koraka, se agenti razdelijo v skupine. Pravilo za deljenje je preprosto in deterministično: Agent  $i$  postane *vodja*, če nobeden od agentov z nižjim ID-jem ne leži znotraj razdalje  $r_{comm}$  (Manhattan). Procesiranje agentov, od najnižjega do najvišjega ID-ja zagotovi konsistentno particijo brez dvomnosti.

**Ključna lastnost:** particija se spremeni vsak korak, tako da je članstvo v skupini dinamično glede na trenutno postavitve.

**2. Željena celica (A\*).** Vsak agent zase neodvisno izvede algoritem A\* od svojega položaja, do cilja pri čimer gleda le na statične ovire. Agent na tej točki ignorira ostale agente. Rezultat tega koraka, je prva celica najkrajše poti, to je celica kamor se agent *želi* premakniti. Agenti, ki so obstali  $\geq t_{esc}$  korakov (*escape mode*), namesto A\* izberejo naključno prosto sosednje celico, s čimer prekinajo tesnobne zastoje v hodnikih.

3. *Prioritetna razrešitev konfliktov.* Agenti se obdelajo v vrstnem redu prioritete:

- (1) agenti v escape modu (najvišja prioriteta),
- (2) vodje skupin,
- (3) sledilci.

Znotraj vsake ravni je prednost pri agentih z nižjim ID-jem. Za vsakega agenta se preverita *vozliščni* in *zamenjalni* konflikt. V primeru konflikta se agent pomakne v najbližjo prosto, še nezaseženo celico. Ker se agenti z višjo prioriteto potrdijo prej, nižjeprioritetni agenti vedno vidijo zasedene celice in se jim izognejo, tako se izognemo konfliktom v fizikalnem koraku.

4. *Posodobitev števca zastoja.* Agent, ki se ni premaknil, poveča števec zastoja; agent, ki se je premaknil, ga ponastavi na nič.

5. *Implementacija.* Algoritem je implementiran v Pythonu za integracijo z okoljem POGEMA in MovingAI, ter v C++ (prek pybind11) za hitrejšo izvajanje. Parametri uporabljeni za testiranje so  $r_{comm} = 7$ ,  $t_{esc} = 8$ .

Lokalni vodja nima globalnega pogleda nad celotnim okoljem, temveč razpolaga le z informacijami o agentih znotraj svoje skupine in njihove neposredne okolice. Na ta način se bistveno zmanjša kompleksnost načrtovanja v primerjavi s centraliziranimi algoritmi, hkrati pa se ohrani višja stopnja koordinacije kot pri popolnoma decentraliziranih pristopih.

Hipoteza našega dela je, da lahko takšen hibridni pristop doseže boljše razmerje med kakovostjo rešitev in časom načrtovanja kot obstoječi centralizirani ali decentralizirani pristopi posebej. Pričakujemo nižji *makespan* in višjo stopnjo uspeha kot pri decentraliziranih metodah ter bistveno krajši čas načrtovanja kot pri popolnoma centraliziranih algoritmi.

## 4 EKSPERIMENT IN ANALIZA REZULTATOV

Po začetni validaciji implementacij smo zasnovali lasten eksperiment, ki bo primerjal centralizirane, decentralizirane in hibridni pristop z lokalnimi vodji. Namen eksperimenta je preveriti ali lahko uvedba lokalnih vodij izboljša razmerje med kakovostjo rešitev in časom načrtovanja.

Experimente izvajamo na standardiziranih zemljevidih iz zbirke MovingAI in POGEMA, na različnih velikostih zemljevidov, gostotah agentov in scenarijih. Tu so najbolj zanimivi scenariji z visoko gostoto agentov, kjer se razlike med rešitvami najbolj pokažejo.

4.0.1 *Testne instance.* Uporabljamo tri glavne tipe okolij:

- **Odperte mreže** (*random grids*) – okolja z malo ovirami, kjer je večina konfliktov posledica visoke gostote agentov.
- **Labirinti** (*mazes*) – okolja z ozkimi prehodi in številnimi lokalnimi ozkimi grli, kjer je koordinacija agentov bistveno zahtevnejša.
- **Skladiščni scenariji** (*warehouse maps*) – realistične postavitve, podobne robotiziranim skladiščem, kjer se agenti pogosto srečujejo na križiščih in v ozkih hodnikih.

Za vsako vrsto okolja izvajamo eksperimente pri različnih številih agentov (od 20 do 1000 agentov, odvisno od velikosti zemljevida), z različnimi konfiguracijami. Vsaka konfiguracija se izvede večkrat

z različnimi scenariji, ki vključujejo začetne in ciljne pozicije agentov, rezultati pa se povprečijo, da zmanjšamo vpliv posameznih naključnih primerov.

4.0.2 *Primerjalni kriteriji.* Primerjavo izvajamo med tremi kategorijami pristopov:

- (1) **Centralizirani pristopi** (LaCAM, PBS),
- (2) **Decentralizirani pristopi** (Follower, MATS-LP),
- (3) **Hibridni pristop** (lokalni vodje).

Za vse metode spremljamo naslednje metrike:

- **SoC**,
- **makespan**,
- **stopnja uspeha**,
- **čas načrtovanja**.

Pri decentraliziranih metodah dodatno spremljamo tudi *throughput*, saj je ta metrika posebej pomembna pri lifelong MAPF scenarijih.

Posebej nas zanima:

- kako hitro začne kakovost centraliziranih metod padati zaradi časovne zahtevnosti,
- pri kateri gostoti agentov decentralizirani pristopi izgubijo stabilnost,
- ali hibridni pristop uspe ohraniti dobro kakovost rešitev ob bistveno krajšem času načrtovanja.

## 4.1 Rezultati

### LITERATURA

- [1] Anton Andreychuk, Konstantin Yakovlev, Alexey Skrynnik, and Aleksandr Panov. 2024. Follower: Learning to Solve Decentralized Lifelong Multi-Agent Pathfinding. In *Proceedings of the AAAI Conference on Artificial Intelligence*. AAAI Press.
- [2] Mehul Damani, Zhiyao Luo, Emerson Wenzel, and Guillaume Sartoretti. 2021. PRIMAL<sub>2</sub>: Pathfinding via Reinforcement and Imitation Multi-Agent Learning – Lifelong. *IEEE Robotics and Automation Letters* 6, 2 (2021), 2666–2673. <https://doi.org/10.1109/LRA.2021.3062803>
- [3] Jiaoyang Li, Daniel Harabor, Peter J. Stuckey, Ariel Felner, Hang Ma, and Sven Koenig. 2019. Disjoint Splitting for Multi-Agent Path Finding with Conflict-Based Search. 29 (Jul. 2019), 279–283. <https://doi.org/10.1609/icaps.v29i1.3487>
- [4] Jiaoyang Li, Wheeler Ruml, and Sven Koenig. 2021. EECBS: A Bounded-Suboptimal Search for Multi-Agent Path Finding. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*. 12353–12362. <https://doi.org/10.1609/aaai.v35i14.17466>
- [5] Keisuke Okumura. 2023. LaCAM: Search-Based Algorithm for Quick Multi-Agent Pathfinding. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 37. AAAI Press, 11655–11662. <https://doi.org/10.1609/aaai.v37i10.26377>
- [6] Alexey Skrynnik, Anton Andreychuk, Anatolii Borzilov, Alexander Chernyavskiy, Konstantin Yakovlev, and Aleksandr Panov. 2025. POGEMA: A Benchmark Platform for Cooperative Multi-Agent Pathfinding. In *The Thirteenth International Conference on Learning Representations*.
- [7] Alexey Skrynnik, Anton Andreychuk, Konstantin Yakovlev, and Aleksandr Panov. 2024. Decentralized Monte Carlo Tree Search for Partially Observable Multi-Agent Pathfinding. In *Proceedings of the Thirty-Eighth AAAI Conference on Artificial Intelligence (AAAI-24)*. AAAI Press.
- [8] Roni Stern, Nathan Sturtevant, Ariel Felner, Sven Koenig, Hang Ma, Thayne Walker, Jiaoyang Li, Dor Atzmon, Liron Cohen, T. Kumar, and Eli Boyarski. 2021. Multi-Agent Pathfinding: Definitions, Variants, and Benchmarks. *Proceedings of the International Symposium on Combinatorial Search* 10 (09 2021), 151–158. <https://doi.org/10.1609/socs.v10i1.18510>
- [9] Yutong Wang, Bairan Xiang, Shao Huang, and Guillaume Sartoretti. 2023. SCRIMP: Scalable Communication for Reinforcement- and Imitation-Learning-Based Multi-Agent Pathfinding. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. <https://doi.org/10.1109/IROS55552.2023.10341379>